Development Environment and Workflow

Predicting Diabetes Risk from Behavioural and Demographic Health Indicators

EE-439 — Introduction to Machine Learning

Faris Mujtaba Qureshi (2022-EE-80) · Mustajib Sami (2022-EE-71)

1. Summary

The entire project was developed locally in Visual Studio Code on Windows 11, using Jupyter notebooks running against a dedicated Conda environment. No cloud notebook service was used, and no part of the analysis depends on one.

2. Tooling

Component	**Detail**
Editor / IDE	Visual Studio Code 1.130.0
Extensions	Python and Jupyter (Microsoft)
Notebook format	Jupyter (.ipynb), executed by the VS Code notebook editor
Kernel	ipykernel 7.3.0
Environment manager	Conda (environment stored inside the project folder)
Language	Python 3.14.6 (CPython)
Version of record	Notebooks are saved with all cell outputs intact

3. Machine used

Component	**Detail**
Operating system	Windows 11 Pro (build 10.0.26200)
Architecture	AMD64 (x86-64)
Processor	Intel 64 Family 6 Model 183
Logical cores	28

Core count affects speed only. Cross-validation and permutation importance are parallelised, so the same code runs correctly but more slowly on a smaller machine. Nothing depends on having 28 cores.

4. Why this setup

- **Local rather than cloud.** The dataset is 21.7 MB and the heaviest model trains in under two seconds, so cloud compute was unnecessary. Working locally also kept package versions fixed for the life of the project.

- **VS Code rather than a bare Jupyter server.** The integrated editor provides version-control awareness, linting and variable inspection beside the notebook, which made debugging the preprocessing much easier.
- **A project-local Conda environment.** Keeping the environment in the project folder rather than installing system-wide means the versions listed in the dependencies document are exactly those the results were produced with.

5. Workflow

Three notebooks run in sequence, each writing files the next reads. Splitting the work this way prevents expensive or irreversible steps from being repeated by accident — in particular the train/test split is made once, in notebook 01, and every later notebook loads that same split rather than making a new one.

Stage	Notebook	Writes
`1. Explore and prepare`	01_data_exploration_and_preprocessing.ipynb	brfss_train_test.npz, brfss_preprocessor.joblib
`2. Train and validate`	02_model_training_and_selection.ipynb	four .joblib models, model_comparison_full.csv
`3. Final testing`	03_final_evaluation.ipynb	final_model_lightgbm.joblib, final_test_results.json

6. Reproducibility

Re-running the notebooks in order reproduces every number in the report exactly. Three things make that true.

- **Fixed random seeds.** Every operation involving randomness — the train/test split, the cross-validation folds, and each model's internal randomness — is seeded with 42.
- **The split is saved, not recomputed.** Notebook 01 writes it to disk and later notebooks load it, so the same records stay on the same side throughout.
- **The fitted preprocessor is saved.** The scaler is fitted on the training set only and stored, so new data can be transformed exactly as the models were trained on.

The test set was opened exactly once, in notebook 03, after the model had already been chosen. It was not used for training, for tuning, for choosing the decision threshold, or for selecting between the four candidates.